Frontend Assignment – Mid-Level (Payment Gateway)

Objective

Design and implement a Payment Gateway UI in Next.js (App Router) with TypeScript. The solution should demonstrate strong frontend fundamentals, clean component design, and the ability to handle real-world payment flow edge cases — without using any third-party payment SDK (Stripe, Razorpay, PayPal, etc.). You will simulate gateway behaviour using a Next.js API Route and manage the full payment lifecycle on the frontend.

Functional Requirements

Payment Form

Build a payment form that collects cardholder name, card number, expiry date (MM/YY), CVV, and amount. The form must validate all fields in real time — errors should appear per field as the user types or on blur, not all at once on submit. The submit button must be disabled until the form is fully valid.

Card Handling

Card number must auto-format with spaces every 4 digits as the user types (e.g. 4242 4242 4242 4242). Detect the card type (Visa, Mastercard, Amex) from the first digits and display a corresponding badge. Expiry must reject past dates. CVV must be 3 digits, or 4 digits for Amex. The amount field must support a currency selector with at least INR and USD.

Card Preview

Display a live card preview that updates as the user fills in the form. The preview should show the card number, cardholder name, and expiry date in a card-like layout.

Payment Lifecycle

The system should support the full payment lifecycle with states: Idle, Processing, Success, Failed, and Timeout. On submit, show a processing state for approximately 2 seconds. After processing, display a distinct result screen for each outcome.

Gateway Simulation

Implement a mock gateway using a Next.js Route Handler at /api/pay. The route should accept a POST request and return one of three outcomes, randomised server-side: Success (approximately 60% of the time), Failed with a reason string such as ‘Insufficient funds’ (approximately 25%), or a delayed response simulating a timeout (approximately 15%, responding after 8 seconds). The frontend must handle a timeout by cancelling the request after 6 seconds using AbortController.

Failure Handling & Retry Logic

On a failed or timed-out payment, offer a retry option. Limit retries to a maximum of 3 attempts per transaction and display the current attempt count to the user (e.g. Attempt 2 of 3). After 3 failed attempts, disable retry and show a final failure message. Each retry must reuse the same transaction ID so that history does not contain duplicate entries.

Transaction History

Maintain a transaction history list that shows each transaction's ID, amount, status, and timestamp. The history must persist across page refreshes using localStorage. The user should be able to click any past transaction to view its details.

Idempotency

Generate a unique transaction ID on the frontend using crypto.randomUUID() before the first attempt. Pass this ID in every request for the same payment so that retries are identifiable and history remains consistent.

Technical Expectations

Code Quality

Write clean, modular, and maintainable code with proper separation of concerns. Components should be split by responsibility — for example CardInput, StatusScreen, and TransactionHistory should each be in separate files. No business logic should live inside JSX; extract validation, formatting, and API calls into utility files or custom hooks. Maintain a clean folder structure: components/, hooks/, utils/, types/, and app/api/.

TypeScript

Use TypeScript throughout with no use of the any type. Define proper interfaces and types for all data structures including PaymentPayload, Transaction, PaymentStatus, and CardType.

State Management

Use Redux Toolkit or Zustand for managing global state across the payment flow. The payment lifecycle (Idle, Processing, Success, Failed, Timeout), transaction history, and any shared UI state must live in the store. Local component state (e.g. form field values) may remain in useState. The choice of library is yours — be prepared to justify it.

Error Handling

Handle network errors separately from API-returned failures. Never expose raw error objects to the user — always show a friendly, readable message. Handle the frontend timeout cleanly using AbortController with a 6-second signal.

Responsiveness & Accessibility

The UI must work correctly on mobile (375px) and desktop (1280px). All form inputs must have visible labels. Error messages must be linked to their inputs using aria-describedby. Focus must be managed correctly after state transitions — the user should never lose their place after a payment result is shown.

Additional Expectations

Demonstrate awareness of real-world payment UX considerations such as preventing double submissions, handling slow networks gracefully, and giving the user clear feedback at every stage of the flow.

Submission

- Push all work to a public GitHub repository before the time limit
- Share the repository link along with any deployment link if available

- Include a README.md with setup instructions, any assumptions made, and what you would improve given more time
- Use meaningful commit messages — at least one commit per completed section